

AWS IAM Explained: Users, Groups, Roles, and Permissions

1. Executive Summary

The core thesis of this video is that **AWS Identity and Access Management (IAM)** is the foundational security service required to safely and effectively interact with AWS resources. The target problem IAM solves is the critical need for granular, centralized access control. By abandoning reliance on the high-risk "Root User" and instead deploying targeted Users, Groups, Roles, and Policies, organizations can securely scale their cloud infrastructure, manage human and application identities, and ensure the principle of least privilege across diverse AWS services like S3 and EC2.

2. Deep-Dive Technical Content

The Root User vs. IAM Users

When an AWS account is first created, a default **Root User** is established. This user has absolute, unrestricted access to the entire AWS environment. The speaker strongly notes that the Root User should only be used initially to create administrative IAM users, after which it should be secured (with MFA) and rarely used again.

IAM Users represent distinct identities (either people or applications) that require access to the AWS environment. Users can be granted two primary types of access:

1. **Console Access:** Password-based login for human interaction with the AWS Management Console.
2. **Programmatic Access:** Key-based authentication (Access Key ID and Secret Access Key) for applications (e.g., a Spring Boot service) interacting with AWS APIs.

Policies: The Engine of Permissions

By default, a newly created IAM user has **zero permissions** (Implicit Deny). Access is explicitly granted via **Policies**—JSON documents that define the actions, resources, and conditions an entity is allowed or denied to execute.

- *Example Edge Case:* If a user lacks an S3 policy, attempting to view or create an S3 bucket results in blanket "API Errors."
- *Implemented Policies in Video:* [AmazonEC2FullAccess](#) and [AmazonS3FullAccess](#).

Groups: Scaling Permission Management

Instead of manually mapping identical policies to multiple developers (e.g., Developer 1, Developer 2, Developer 3), which introduces massive operational overhead and drift, engineers should utilize **IAM Groups**.

- Groups are collections of IAM users.
- Policies attached to the Group are inherently inherited by all member Users.
- *Video Example:* A [Developer](#) group was assigned [EC2FullAccess](#), [S3FullAccess](#), and [AWSLambda_FullAccess](#). Any user added to this group instantly gains these permissions.

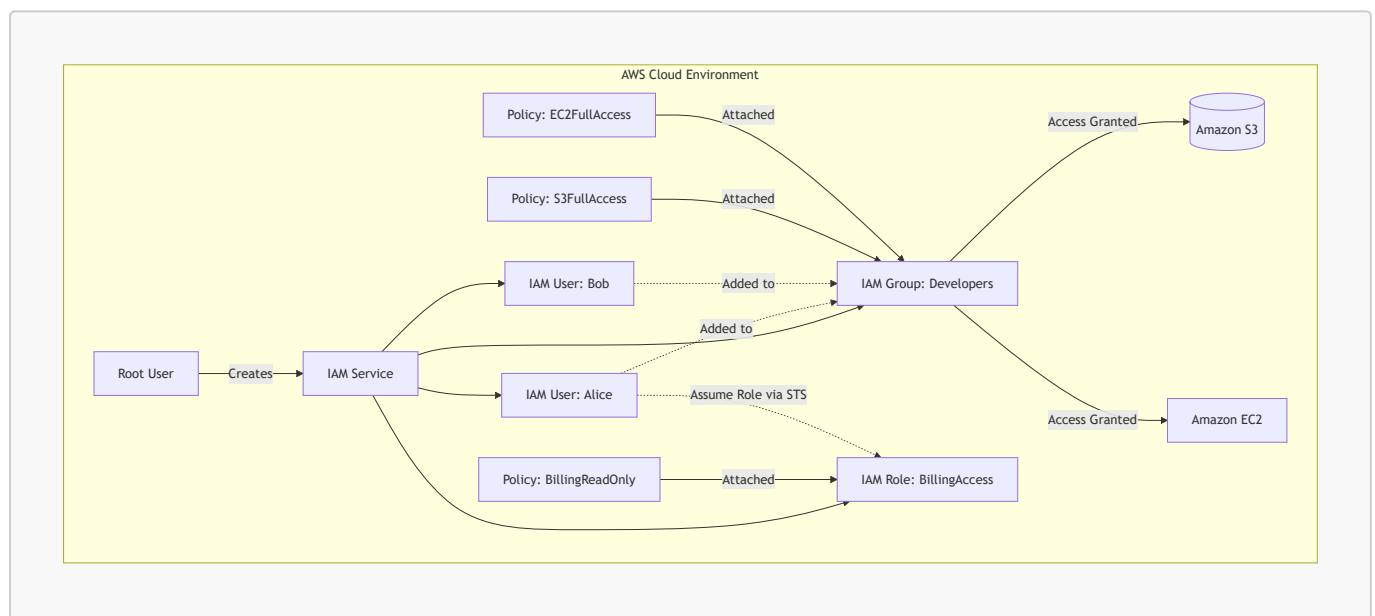
Roles and the `sts:AssumeRole` Action

An **IAM Role** is fundamentally different from a User; it does not have standard long-term credentials (passwords or access keys). Instead, a Role is a set of permissions that can be **assumed temporarily** by anyone (or anything) that has been explicitly trusted.

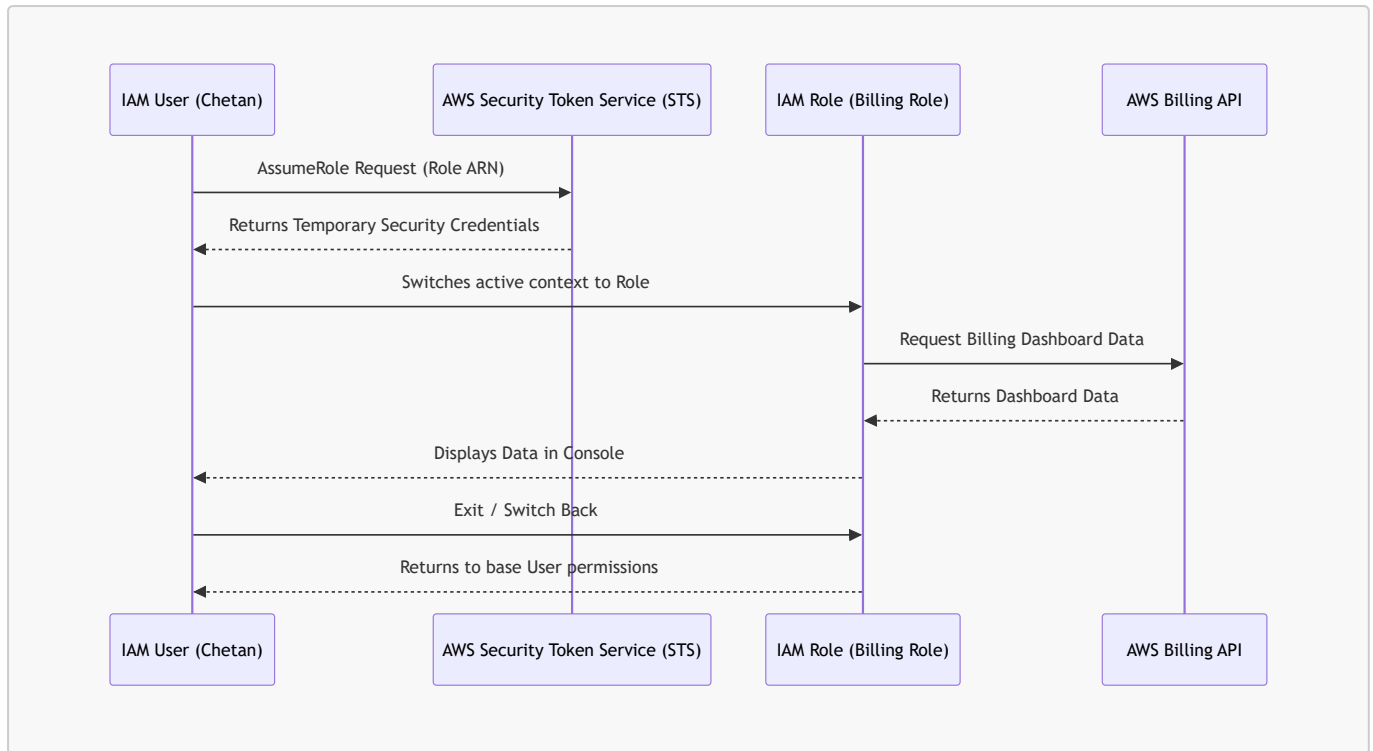
- *Use Case Discussed:* A developer needs temporary access to AWS Billing dashboards but should not have permanent read access.
- *Mechanism:* A "Billing Role" is created with the `AWSBillingReadOnlyAccess` policy. The target user is given an inline policy granting `sts:AssumeRole` targeted at the Billing Role's Amazon Resource Name (ARN). The user triggers a "Switch Role" action in the console, temporarily trading their base permissions for the Role's permissions.

3. Code & Architecture Visualizations

Architecture: IAM Component Relationship



Architecture: Role Assumption Data Flow



Code Implementation: Programmatic Access (Java/Spring Boot)

The speaker notes that application users connect programmatically. Below is the Java equivalent for implementing programmatic access to S3 using the IAM Access Keys discussed in the video.

```

import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.CreateBucketRequest;

/**
 * Service to connect to AWS S3 using Programmatic IAM Credentials.
 * In a production Spring Boot application, these credentials should
 * be injected via environment variables or Secrets Manager.
 */
public class AwsS3Service {

    public void createBucketProgrammatically(String accessKey, String secretKey,
String bucketName) {

        // 1. Define the IAM Programmatic Credentials
        AwsBasicCredentials credentials = AwsBasicCredentials.create(accessKey,
secretKey);

        // 2. Build the S3 Client securely injecting the IAM identity
        S3Client s3Client = S3Client.builder()
            .region(Region.US_EAST_1)

            .credentialsProvider(StaticCredentialsProvider.create(credentials))
            .build();
    }
}

```

```
try {
    // 3. Execute the API call (Requires AmazonS3FullAccess policy as
    shown in the video)
    CreateBucketRequest bucketRequest = CreateBucketRequest.builder()
        .bucket(bucketName)
        .build();

    s3Client.createBucket(bucketRequest);
    System.out.println("Bucket created successfully via Programmatic IAM
Access: " + bucketName);

} catch (Exception e) {
    // Will throw an exception if the IAM user lacks explicit S3 policies
    System.err.println("API Error: Missing permissions or bucket
collision. " + e.getMessage());
} finally {
    s3Client.close();
}
}
```

Code Implementation: STS Assume Role Policy (JSON)

To allow an IAM user to switch roles to the Billing dashboard, they must have an inline policy specifically granting the `sts:AssumeRole` action targeting the exact Role ARN.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowBillingRoleAssumption",
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::123456789012:role/billing-role"
    }
  ]
}
```

4. Key Metrics & Comparisons

Authentication & Entity Types

Feature	IAM User	IAM Group	IAM Role	Root User
Primary Use Case	Long-term credentials for individual people or apps.	Managing permissions logically for multiple users.	Temporary, short-lived permissions for cross-account or app access.	Account creation, emergency account recovery.
Credentials Used	Passwords (Console) / Access Keys (API).	N/A (Cannot authenticate as a group).	STS Temporary Tokens (No long-term keys).	Email/Password combination.
Best Practice	Use for daily tasks. Require MFA.	Use to assign policies instead of direct User assignment.	Use for application logic (e.g., EC2 connecting to S3).	Do not use for daily operations.

Access Modalities

Access Type	Target Audience	Authentication Mechanism	Pros	Cons
Management Console	Human Engineers, Administrators	Username + Custom Password	Easy GUI, fast navigation, visual feedback.	Not scalable for automation; UI changes frequently.
Programmatic / CLI	CI/CD Pipelines, Microservices (Spring Boot)	Access Key ID + Secret Key	Highly scalable, enables Infrastructure as Code (IaC).	High security risk if keys are leaked/committed to GitHub.

5. Actionable Takeaways

- **Never Use the Root Account for Daily Operations:** Secure the root account with a strong password and multi-factor authentication (MFA), and immediately create admin-level IAM users for configuration tasks.
- **Adopt IAM Groups for Scalability:** Never attach permission policies directly to individual human users. Always attach policies to Groups (e.g., **Developers**, **QA**, **DevOps**) and assign users to those groups to prevent administrative overhead and permission drift.
- **Implement the Principle of Least Privilege:** Start all IAM users and roles with zero permissions. Only add explicit **Allow** policies (e.g., **S3FullAccess** or **EC2FullAccess**) when absolutely necessary for a task.
- **Leverage IAM Roles for Cross-Boundary Access:** Instead of distributing permanent access keys, use **sts:AssumeRole** configurations. This allows users or microservices to request temporary, auto-expiring credentials to perform sensitive tasks (like checking billing).
- **Verify Edge Case Permissions:** A general "Billing Access" policy may miss specific sub-features. (As noted in the video, **AWSCostAndUsageReport** had to be explicitly added to allow full visibility of the Cost Summary).